Sketch-Prompted VLA

A Learned Text-Only Baseline:
$\pi_{0.5}$-LIBERO on the Three Validation Suites

Aaron

6 August 2026

Abstract

The 114 validation scenes of this project are ambiguous by construction: the instruction alone does not identify which of several identical candidates is meant. Until now the only policies run on them have been scripted — a sketch-reading oracle and a firewalled text-only guesser — which establish that the harness scores correctly but say nothing about how a real vision-language-action model behaves under that ambiguity. This report supplies that missing measurement. I evaluate $\pi_{0.5}$-LIBERO, Physical Intelligence's flow-matching VLA fine-tuned on the four standard LIBERO suites, as a *learned, sketch-free* baseline: the stock checkpoint, fed the same instruction string every sketch condition receives, with no modification to the model. I first reproduce the published benchmark numbers to certify the observation pipeline, then report success on my own scenes, separating the two causes of the gap — distribution shift away from the standard suites, and referential ambiguity within my tiers. **$\pi_{0.5}$-LIBERO scores 34.5% over 342 rollouts on the 114 scenes, against 96.0% from the same pipeline, the same checkpoint and the same text-only input on the standard suites — the two differ in the scenes, not in what the model is shown, and no sketch is involved on either side. The control tier — unambiguous instruction, one candidate — scores 73.3%, the other three tiers 28.6%. Distribution shift therefore costs roughly 25 points and referential ambiguity roughly 45; the second is the headroom a sketch is meant to recover, and it is the larger of the two.**

1 Why $\pi_{0.5}$, and why nothing is ablated

$\pi_{0.5}$ consumes a base image, a wrist image, an eight-dimensional proprioceptive state and a language string, and emits an action chunk. It has no sketch channel. "$\pi_{0.5}$ without the sketch" therefore requires no edit to the model, the config or the checkpoint: the sketch-free baseline *is* the stock checkpoint driven by `meta['instruction']`. My claim is that this makes it the right baseline rather than merely a convenient one — there is no ablation whose faithfulness could be disputed, and the number it produces is the number a competent practitioner would get from the strongest openly available LIBERO policy with no knowledge of this project.

The checkpoint is `gs://openpi-assets/checkpoints/pi05_libero`, served by openpi's policy server at config `pi05_libero`. Model and simulator are kept in separate processes communicating over a websocket, which is how openpi ships it; that split also means the LIBERO environment used here is byte-identical to the one the scripted policies run in.

2 Certifying the observation pipeline

Four preprocessing details — render resolution, a 180° rotation, the wrist camera, and chunked action execution — each degrade the success rate silently rather than raising an error. A low number on my scenes is only interpretable once they are known to be right, so the published suites are reproduced first.

Table 1: Reproduction of the published $\pi_{0.5}$-LIBERO numbers, using openpi's own unmodified `examples/libero/main.py`. Published figures are 500 trials per suite; my reproduction uses 5 trials per task (50 episodes per suite), sufficient to separate a working pipeline from a broken one.

Suite	Published (%)	Observed (%)	Trials/task
LIBERO-Spatial	98.8	98.0	5
LIBERO-Object	98.2	100.0	5
LIBERO-Goal	98.0	96.0	5
LIBERO-10	92.4	90.0	5
Average	96.85	96.0	

It reproduces, and nothing needed adjusting. Every suite lands within 2.4 points of its published figure — worst case LIBERO-10 at 90.0 against 92.4 — and the average is 96.0 against 96.85, on a fortieth of the trial count. At 50 episodes per suite the binomial standard error is about 3 points, so the residual differences are not distinguishable from sampling noise, and Object landing at 100.0 above its published 98.2 is the expected behaviour of a small sample rather than evidence of anything. The install, the checkpoint and the reference preprocessing are therefore certified before any number off my own scenes is believed. Full detail, including the openpi commit (`15a9616`), is in `outputs/rollouts/pi05_baseline/openpi_reference.json`.

2.1 The 180° rotation, settled by measurement

`SUITE_FACTS.md` states that `obs['agentview_image']` is already correctly oriented and must not be flipped, while openpi applies `img[::-1,::-1]` to both cameras. Both are true and they are about different things — the first governs the *projection* path, where pixels must line up with `frame0.png`; the second governs what the *model* is fed — but that was reasoning from source, not a measurement. I settled it by dumping the exact array the policy hands to the server (`-pi05-dump-frame`) and correlating it against each scene's `frame0.png`. The dumped frame matches the 180°-rotated `frame0` at normalised cross-correlation 0.977–0.995 across the three smoke scenes, and *anti*-correlates with `frame0` as-is (-0.23 to -0.51). Figure 1 shows the three panels.

The rotation is also the thing that matters, not merely the thing that differs. Rerunning the same three scenes with `-pi05-no-rotate180` takes the policy from grasping the correct object in 3 of 3 rollouts to grasping *nothing at all* in 3 of 3 — an unrecoverable collapse rather than a degradation.

2.2 Divergences from openpi's reference loop

Two, both deliberate and both recorded in `run_config.json`:

- **No settling wait.** openpi steps ten dummy actions at episode start to let dropped objects come to rest. My `init_state.npz` was pinned *after* settling and is the state the sketch was drawn on, so every condition starts from the identical already-settled configuration and the wait is set to zero.
- **Render resolution is decoupled from the sketch canvas.** Scenes are rendered at 128 for sketching and at 256 for this policy, from the same BDDL and the same restored simulator state. The restored state carries no camera information, so the initial physical configuration is identical across the two — which is what makes the conditions comparable.

A third difference is not a divergence in intent but is worth recording, because it makes my numbers marginally *harsher* than openpi's: openpi breaks the episode the moment the environment reports success, whereas this harness always runs the full step budget and scores `success_sustained` — the BDDL goal predicate holding over a five-step window. A policy that reaches the goal and then knocks the object off before the window closes is scored a failure here and a success there.

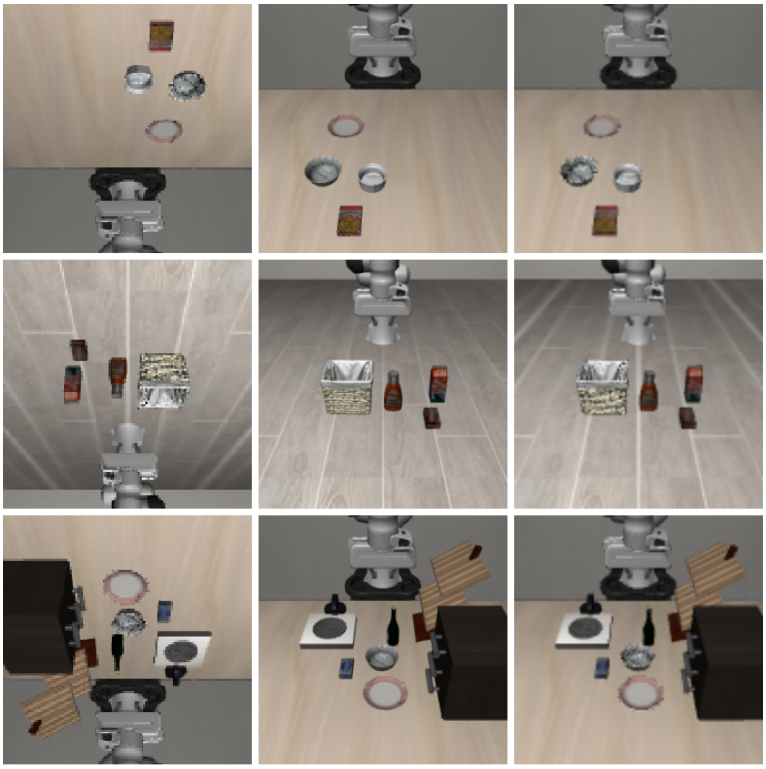

Figure 1: One row per smoke scene (spatial, object, goal). **Left:** `frame0.png`, the 128 px sketch canvas, in the orientation the projection path requires. **Middle:** the exact 224 px array the model received. **Right:** 180°-rotated `frame0` pad-resized to 224, the reference the middle column is compared against.

3 Results on the validation suites

Table 2: $\pi_{0.5}$-LIBERO, text-only, on the 114 validation scenes, three rollouts per scene. Sustained success is the harness's own criterion: the BDDL goal predicate holding over a five-step window, from `env.check_success()`. "Correct object" is `correct_instance_grasped`. "Correct destination" is the harness's *approximate* post-hoc xy-proximity attribution, not a goal predicate — see limitations.

Suite	Scenes	Rollouts	Sustained success (%)	Correct object (%)	Correct destination (%)
Spatial	38	114	24.6	58.8	50.0
Object	38	114	27.2	45.6	61.4
Goal	38	114	51.8	88.6	25.4
All	114	342	34.5	64.3	45.6

The number to quote is **34.5%**, over all 114 scenes and all 342 rollouts, with nothing skipped. A large gap from the published 96.85% was the expected result and is not a criticism of the checkpoint: these are synthesised BDDL scenes with novel object arrangements and deliberately ambiguous captions, and the gap is the headroom the sketch is supposed to recover.

One reading of that gap must be ruled out explicitly, because it is the easiest mistake to make with these two numbers. It is *not* a with-sketch versus without-sketch comparison. $\pi_{0.5}$ has no sketch channel, so no drawing enters either measurement: both sides are the stock checkpoint fed an image pair, a state vector and a language string, and both are therefore text-only. What changes between 96.0% and 34.5% is the scenes — familiar suites the checkpoint was fine-tuned on, each with one right answer, against synthesised scenes whose captions do not identify the referent. Attributing the drop to a missing sketch would credit the sketch with a recovery this report does not measure and that no run in this repository has yet measured on a learned policy.

3.1 Separating distribution shift from ambiguity

The gap has two causes, and conflating them would overstate what a sketch can recover. The control tier separates them: it carries an unambiguous instruction with exactly one candidate, so nothing about it is ambiguous and whatever it loses relative to the standard suites is distribution shift alone.

Table 3: Success by tier. Chance is computed per scene from the number of same-category candidates the caption cannot distinguish ($1/k$ for the object, $1/m$ for the destination), averaged over the block — the floor below which no text-only policy can be blamed.

Tier	Scenes	Rollouts	Sustained success (%)	Correct object (%)	chance (%)	
					object	destination
Control	15	45	73.3	86.7	100.0	100.0
Referential	36	108	38.0	50.0	34.8	100.0
Directional	27	81	33.3	85.2	100.0	39.8
Both	36	108	15.7	53.7	31.1	42.6
Other tiers	99	297	28.6	60.9		

The split, measured against my own reproduction rather than the published figure so that both sides come from the same pipeline:

- **Distribution shift ≈ 24.7 points.** The same pipeline scores 98.0% averaged over the three standard suites my scenes derive from (Spatial, Object, Goal); the control tier scores 73.3%. Nothing in a control scene is ambiguous, so this is the cost of novel arrangements and synthesised BDDL alone.
- **Referential ambiguity ≈ 44.7 points.** Control 73.3% against the other three tiers at 28.6%. This is the part a sketch prompt addresses, and it is the larger of the two.

Two tier-level results sharpen this beyond the headline. On the **directional** tier — one candidate object, several plausible destinations — the policy identifies the object almost perfectly (85.2% correct, against a ceiling of 100%) and then delivers to the right destination **40.7% of the time against a chance level of 39.8%**. That is chance, to within a point. The model is not weakly disambiguating destinations; on these scenes it is not disambiguating them at all, which is exactly what one expects when the caption does not contain the information. On the **referential** tier the picture is different: 50.0% correct object against 34.8% chance, so the model is meaningfully above chance but far from resolved — some captions carry weak spatial cues it can exploit, and it exploits them partially.

3.2 Failure modes

The harness distinguishes "grasped the wrong object" from "failed to manipulate at all" via `correct_instance_grasped` and `grasped_any`. That distinction is what makes the headline number mean something, and here it is unusually clean: $\pi_{0.5}$ **lifted something in 97.1% of all 342 rollouts.** Failure to manipulate is essentially never the blocker. Of the 65.5% of rollouts that failed, only 1.2 points involved no grasp at all; 32.5 points grasped the wrong object and 31.9 points grasped the right object and still did not complete the placement.

Of the 112 wrong-object grasps, **73.2% were a same-category sibling** — one of the near-identical duplicates the scene was built around — and 26.8% were some other object. The dominant failure is therefore referential confusion of exactly the kind the circle is designed to remove, not perceptual confusion between unlike objects.

The Goal suite is the outlier in Table 2 and the reason is visible in the same columns: it has the highest correct-object rate (88.6%) and by far the lowest correct-destination rate (25.4%). Goal

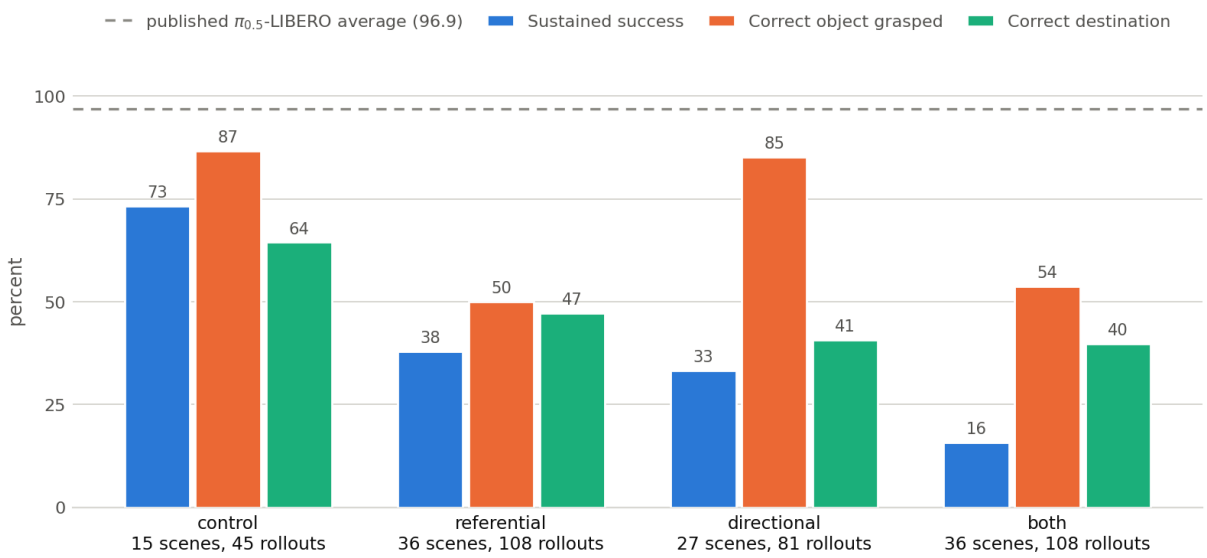

Figure 2: Success and failure-mode breakdown by tier, 114 scenes × 3 rollouts. Correct-object accuracy stays high wherever the *object* is unambiguous (control, directional) and collapses wherever it is not (referential, both), while sustained success falls monotonically with the number of ambiguous choices the caption leaves open.

scenes frequently use a *region*-typed destination — a named patch of table with no object at it — and the proximity proxy cannot credit a delivery to a region it has no body to measure against, so 25.4% understates real destination accuracy there. Its sustained-success figure of 51.8% comes from `check_success()` and is not affected by this.

4 Limitations

- $\pi_{0.5}$-LIBERO was fine-tuned on the four standard suites; my scenes are synthesised BDDL with novel arrangements. Its performance here is not a measure of the model's quality, and no attempt is made to tune it.
- The instruction strings in the non-control tiers are ambiguous by design. A perfect policy cannot exceed chance on the referent it is not told about, so a portion of the failure rate is irreducible for *any* text-only policy — that portion is precisely what the chance columns in Table 3 quantify.
- **The 24.7 / 44.7 split rests on 15 control scenes and 45 rollouts.** That is the whole control tier, but it is a small denominator: the binomial standard error on 73.3% at $n = 45$ is about 6.6 points, and the three rollouts within a scene are not independent of each other, so the true interval is wider still. The ordering of the two causes is robust; the precise point values are not.
- **`correct_destination` is an approximate xy-proximity proxy** (`WRONG_DEST_APPROX_TH = 0.08m`), not LIBERO's contact solver, and it is systematically pessimistic for region-typed destinations as noted above. Only `success_sustained` comes from `env.check_success()`, and only it should be quoted as a success rate.
- **The reproduction is 50 episodes per suite, not 500.** It is sized to separate a working pipeline from a broken one, which is all it is asked to do; it is not a re-measurement of the published figures.
- The chance levels in Table 3 assume a policy that has correctly identified the target *category* and is guessing uniformly among its instances. A policy that has not identified the category can score below them, which is what the 26.8% of wrong grasps landing on non-siblings

shows.

- The run was executed as two invocations sharing one `-run-id` with `-resume` — 36 scenes, then all 114 — which is the harness's normal resumption path and produced 342 rows with zero skips and zero duplicates.

5 Reproducing this

- Brief: `prompt_pi05_baseline.md`.
- Policy: `scripts/pi05_policy.py`; harness path `scripts/rollout_sketch_wsl.py -policy pi05`.
- Run artefacts: `outputs/rollouts/pi05_baseline/` (`results.csv`, `run_config.json`, `summary.json`, `analysis.json`, `openpi_reference.json`, `videos/`).
- Orientation check: `outputs/rollouts/pi05_smoke/model_input/` and the no-rotation control in `outputs/rollouts/pi05_smoke_norot/`.
- openpi at `main`, commit `15a9616a` (not `kevin/pi05-support`, which predates the merge); SHA recorded in `openpi_reference.json`.
- Cost of the measurement: 19,608 inference calls at 86.6 ms mean latency, roughly 50 minutes of wall clock on one RTX 4090 for the 114-scene run.